La riducibilità è un metodo per convertire l'istanza di un problema A in un'istanza di un problema B e utilizzare la soluzione di quest'ultimo per ottenere la soluzione di A. La riducibilità è anche uno dei metodi principali per dimostrare che un problema non è computazionalmente decidibile.

Quando A è riducibile a B:
- Risolvere A non può essere più difficile di risolvere B;
- Risolvere B ci permette di ottenere la soluzione di A;
- Se B è decidibile allora anche A lo è;
- Se A non è decidibile allora anche B è non decidibile;

5.1 PROBLEMI INDECIDIBILI DALLA TEORIA DEI LINGUAGGI

Abbiamo già stabilito l'indecidibilità di A_{TM}, il problema di determinare se una macchina di Turing accetta un dato input. Consideriamo ora un problema affine, $HALT_{TM}$: un problema di determinare se una macchina di Turing si ferma (accettando o rifiutando) su un dato input. Questo problema è generalmente noto come il problema della fermata. Usiamo l'indecidibilità di A_{TM} per dimostrare l'indecidibilità del problema della fermata riducendo A_{TM} a $HALT_{TM}$. Sia: $HALT_{TM} = \{\langle M, w\rangle \mid M \text{ è una MdT e } M \text{ si ferma su input } w\}$

Teorema 5.1
$HALT_{TM}$ è indecidibile.

IDEA: Questa dimostrazione è per assurdo. Assumiamo che $HALT_{TM}$ è decidibile ed utilizziamo questa assunzione per dimostrare che A_{TM} è decidibile, contraddicendo il teorema 4.11. L'idea chiave è quella di mostrare che A_{TM} è riducibile ad $HALT_{TM}$.

Supponiamo di avere una MdT R che decide $HALT_{TM}$. Usiamo quindi R per costruire S, una MdT che decide A_{TM}. S riceve un input nella forma $\langle M, w\rangle$. Da in output "accetta" se M accetta w, e da in output "rifiuta" se M cicla oppure rifiuta w. S però potrebbe non essere in grado di determinare se M è entrata in un ciclo, in questo caso la simulazione non terminerà. Questo non va bene perché S è un decisore e non gli è permesso ciclare. Questa idea quindi non funziona.

Utilizziamo invece l'ipotesi che S possiede una MdT R che decide $HALT_{TM}$. Con R, S può verificare se M si ferma su w. Se R indica che M non si ferma su w, rifiutate perché $\langle M, w\rangle$ non è in A_{TM}. In caso contrario, S può fare la simulazione senza alcun pericolo di ciclare. Così, se la MdT R esistesse, potremmo decidere A_{TM}, ma sappiamo che A_{TM} è indecidibile. In virtù di questa contraddizione possiamo concludere che R non esiste. Pertanto $HALT_{TM}$ è indecidibile.

DIM: Assumiamo alla fine di ottenere una contraddizione che la MdT R decide $HALT_{TM}$. Costruiamo la MdT S per decidere A_{TM}, che opera come segue.

S = "Su input $\langle M, w\rangle$, una codifica di una MdT M e una stringa w:
1. Esegue la MdT R su input $\langle M, w\rangle$;
2. Se R rifiuta, rifiuta;
3. Se R accetta, simula M su w finché non si ferma;
4. Se M ha accettato, accetta; se M ha rifiutato, rifiuta."

Chiaramente, se R decide $HALT_{TM}$, allora S decide A_{TM}. Poiché A_{TM} è indecidibile, allora anche $HALT_{TM}$ deve essere indecidibile.

Presentiamo ora vari altri teoremi e le loro dimostrazioni come ulteriori esempi dell'uso della riducibilità per dimostrare l'indecidibilità. Sia $E_{TM} = \{\langle M\rangle \mid M \text{ è una MdT e } L(M) = \emptyset\}$.

/ /

Teorema 5.2

E_{TM} è indecidibile.

IDEA: Assumiamo che E_{TM} è decidibile e mostriamo che A_{TM} è decidibile - una contraddizione. Sia R una MdT che decide E_{TM}. Utilizziamo R per costruire la MdT S che decide A_{TM}. Come funzionerà S quando riceve in input $\langle M, w \rangle$?

Un'idea è che S esegue R su input $\langle M \rangle$ e vede se R accetta. Se lo fa sappiamo che $L(M)$ è vuoto e quindi che M non accetta w. Ma, se R rifiuta $\langle M \rangle$, tutto quello che sappiamo è che $L(M)$ non è vuoto e, di conseguenza, che M accetta qualche stringa - ma ancora non sappiamo se M accetta w in particolare. Quindi abbiamo bisogno di un'idea diversa.

Invece di eseguire R su $\langle M \rangle$, eseguiamo R su una modifica di $\langle M \rangle$. Modifichiamo $\langle M \rangle$ così da garantire che M rifiuta tutte le stringhe tranne w, ma su input w, funziona come al solito. A questo punto, utilizziamo R per determinare se la macchina modificata riconosce il linguaggio vuoto. L'unica stringa che adesso la macchina può accettare è w, per cui il suo linguaggio sarà non vuoto se accetta w. Se R accetta quando riceve in input la descrizione della macchina modificata, sappiamo che la macchina modificata non accetta nulla e che M non accetta w.

DIM: Descriviamo la macchina modificata M_1:

M_1 = "su input x:

1. Se $x \neq w$, rifiuta;
2. Se $x = w$, esegue M su input w e accetta se M accetta."

Mettendo insieme tutto questo, assumiamo che la MdT R decide E_{TM} e costruiamo la MdT S che decide A_{TM} come segue:

S = "su input $\langle M, w \rangle$, una codifica di una MdT M e una stringa w:

1. Usa la descrizione di M e w per costruire la MdT M_1 descritta sopra;
2. Esegue R su input $\langle M_1 \rangle$;
3. Se R accetta, rifiuta; se R rifiuta, accetta".

Se R fosse un decisore di E_{TM}, S sarebbe un decisore per A_{TM}. Un decisore per A_{TM} non può esistere, quindi sappiamo che E_{TM} deve essere indecidibile.

Un altro interessante problema computazionale che concerne le macchine di Turing è quello di determinare se una data MdT riconosce un linguaggio che può essere riconosciuto anche da un modello di calcolo più semplice. Ad esempio, sia $REGULAR_{TM}$ il problema di determinare se una data MdT ha un automa finito equivalente.

Questo problema equivale a determinare se la MdT riconosce un linguaggio regolare. Sia:

$REGULAR_{TM} = \{\langle M \rangle / M \text{ è una MdT ed } L(M) \text{ è un linguaggio regolare}\}$

Teorema 5.3

$REGULAR_{TM}$ è indecidibile

IDEA: Come al solito per i teoremi sull'indecidibilità, questa dimostrazione consiste in una riduzione da A_{TM}.
Assumiamo che $REGULAR_{TM}$ sia deciso da una MdT R e utilizziamo questa assunzione per costruire una MdT S che decide A_{TM}.

L'idea è che S prenda il suo input $\langle M, w \rangle$ e modifichi M in modo che la risultante MdT riconosca un linguaggio regolare se e solo se M accetta w. Chiamiamo M_2 la macchina così modificata. Progettiamo M_2 in modo che riconosca il linguaggio non regolare $\{0^n 1^n / n \geq 0\}$ se M non accetta w, e riconosca il linguaggio regolare Σ^* se M accetta w. Dobbiamo specificare come S può costruire una tale M_2 da M e w. Qui, M_2 accetta automaticamente tutte le stringhe in $\{0^n 1^n / n \geq 0\}$. Inoltre se M accetta w, M_2 accetta tutte le altre stringhe.

Notate che la MdT M_2 non è costruita con lo scopo di essere eseguita su qualche input. Costruiamo M_2 al solo scopo di dare in input la sua descrizione al decisore per $REGULAR_{TM}$ che abbiamo assunto esistere. Quando tale decisore dà la sua risposta, possiamo usarla per rispondere se M accetta w o meno. Quindi possiamo decidere A_{TM}, una contraddizione.

DIM: Definiamo R come una MdT che decide $REGULAR_{TM}$ e costruiamo una MdT S che decide A_{TM}. Allora S funziona come segue:

S = "Su input $\langle M, w \rangle$, dove M è una MdT e w è una stringa:
1. Costruisce la seguente MdT M_2.
   M_2 = "Su input x:
   1. Se x ha forma 0^n1^n, accetta;
   2. Se x non ha tale forma, esegue M su input w e accetta se M accetta w."
2. Esegue R su input $\langle M_2 \rangle$.
3. Se R accetta, accetta; se R rifiuta, rifiuta".

Finora la nostra strategia per dimostrare l'indecidibilità di linguaggi prevede una riduzione da A_{TM}. A volte la riduzione da qualche altro linguaggio indecidibile, come ad esempio E_{TM}, risulta più conveniente per dimostrare che certi linguaggi sono indecidibili. Il teorema 5.4 mostra che verificare l'equivalenza di due macchine di Turing è un problema indecidibile. Potremmo provarlo con una riduzione da A_{TM}, ma è più conveniente una riduzione da E_{TM}. Sia:
$EQ_{TM} = \{\langle M_1, M_2 \rangle \mid M_1 \text{ e } M_2 \text{ sono MdT ed } L(M_1) = L(M_2)\}$

Teorema 5.4
EQ_{TM} è indecidibile.

IDEA: Mostriamo che se EQ_{TM} fosse decidibile allora anche E_{TM} sarebbe decidibile (contraddizione) eseguendo una riduzione da E_{TM} a EQ_{TM}. L'idea è semplice. E_{TM} è il problema di determinare se il linguaggio di una MdT è vuoto. EQ_{TM} è il problema di determinare se i linguaggi delle due MdT sono uguali. Se uno di questi linguaggi è $\emptyset$, ci ritroviamo con il problema di determinare se il linguaggio dell'altra macchina è vuoto. Quindi, in un certo senso, il problema di E_{TM} è un caso particolare del problema EQ_{TM} dove una delle macchine è fissata per riconoscere il linguaggio vuoto. Questa idea rende la riduzione più semplice.

DIM: Consideriamo una MdT R che decide EQ_{TM} e una MdT S che decide E_{TM} come segue:
S = "Su input $\langle M \rangle$, dove M è una MdT:
1. Esegue R su input $\langle M, M_1 \rangle$, dove M_1 è una MdT che rifiuta ogni input.
2. Se R accetta, accetta; se R rifiuta, rifiuta".

Se R decide EQ_{TM}, S decide E_{TM}. Ma il teorema 5.2 ci dice che E_{TM} è indecidibile, quindi anche EQ_{TM} deve essere indecidibile.

RIDUZIONE MEDIANTE STORIE DI COMPUTAZIONE

Il metodo mediante storie di computazione è una tecnica importante per dimostrare che A_{TM} è riducibile a certi linguaggi. Questo metodo è spesso utile quando il problema da mostrare indecidibile comporta la dimostrazione dell'esistenza di qualcosa. La computazione per una macchina di Turing su un input è semplicemente la sequenza delle configurazioni attraverso cui la macchina passa mentre elabora l'input. Si tratta di una registrazione completa della computazione della macchina.

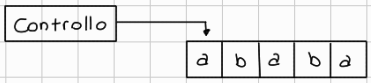
Definizione 5.5
Sia M una MdT e w una stringa in input. Una storia di computazione accettante per M su w è una sequenza di configurazioni $C_1, C_2, \ldots, C_l$, dove C_1 è la configurazione iniziale di M su w, C_l è una configurazione di accettazione per M, e ogni C_i segue da C_{i-1} in accordo alle regole di M. Una storia di computazione di rifiuto per M su w è definita in modo analogo, tranne per il fatto che C_l è una configurazione di rifiuto.

Le storie di computazione sono sequenze finite. Se M non si ferma su w, non esiste una storia di computazione accettante o di rifiuto per M su w. Le macchine deterministiche hanno al massimo una storia di computazione su ogni input. Le macchine non deterministiche possono avere più storie di computazione su un singolo input, corrispondenti ai diversi rami di computazione.

Definizione 5.6

Un automa linearmente limitato è un tipo ristretto di MdT in cui alla testina del nastro non è permesso di spostarsi fuori dalla parte del nastro che contiene l'input. Se la macchina tenta di spostare la testina al di fuori dell'input, la testina rimane dove si trova - come accade in una normale MdT dove la testina non avanza oltre l'estremità sinistra del nastro.

Un automa linearmente limitato è una MdT con una quantità limitata di memoria, come mostrato schematicamente nella figura. Può risolvere solo problemi che richiedono una memoria al più pari alla dimensione dell'input. L'uso di un alfabeto di nastro più grande dell'alfabeto di input consente alla memoria disponibile di essere aumentata di un fattore costante. Quindi diciamo che per un input di lunghezza n, la quantità di memoria disponibile è lineare in n - da qui il nome di questo modello.



Nonostante il vincolo sulla memoria, un automa linearmente limitato (LBA) è abbastanza potente. Ad esempio, i decisori per A_{DFA}, A_{CFG}, E_{DFA} e E_{CFG} sono tutti LBA. Ogni CFL può essere deciso da un LBA.

Qui A_{LBA} è il problema di determinare se un LBA accetta il suo input. Sia:
$A_{LBA} = \{\langle M, w\rangle \mid M \text{ è un LBA che accetta la stringa } w\}$

Prima di dimostrare la decidibilità di A_{LBA}, troviamo utile il seguente lemma. Esso dice che un LBA può avere solo un numero limitato di configurazioni quando ha in input una stringa n.

Lemma 5.8

Sia M un LBA con q stati e g simboli nell'alfabeto di nastro. Esistono esattamente qng^n configurazioni distinte di M per un nastro di lunghezza n.

DIM: Ricordiamo che una configurazione di M è come un'istantanea durante il suo calcolo. Una configurazione comprende lo stato del controllo, la posizione della testina ed il contenuto del nastro. Qui M ha q stati. La lunghezza del suo nastro è n, per cui la testina può essere in una delle n posizioni, e g^n possibili stringhe di simboli del nastro compaiono sul nastro. Il prodotto di queste tre quantità è il numero totale di configurazioni differenti di M con un nastro di lunghezza n.

Teorema 5.9

A_{LBA} è decidibile

IDEA: Per decidere se un LBA M accetta l'input w, simuliamo M su w. Nel corso della simulazione, se M si ferma e accetta o rifiuta, accettiamo o rifiutiamo di conseguenza. La difficoltà si presenta se M cicla su w. Dobbiamo essere in grado di rilevare tali situazioni in modo da poter fermare e rifiutare. L'idea per rilevare se M cicla è che, mentre M computa su w, essa passa da configurazione a configurazione. Se M ripetesse una configurazione essa continuerebbe a ripeterla più volte, quindi ciclerebbe. Il Lemma 5.8, ci dice che M può essere in un numero limitato di configurazioni su questa quantità di nastro. Rilevare che è entrata in un ciclo è possibile simulando M per il numero di passi dati dal Lemma 5.8. Se M non si ferma entro tale numero di passi, allora deve essere entrata in un ciclo.

DIM: L'algoritmo che decide A_{LBA} funziona come segue:

L = "su input $\langle M, w \rangle$, dove M è un LBA e w è una stringa:

1. Simula M su w per qng^n passi finché non si ferma;
2. Se M si è fermata, accetta se ha accettato e rifiuta se ha rifiutato. Se non si è fermata, rifiuta".

Il teorema 5.9 mostra che LBA e MdT differiscono in maniera essenziale: per i primi il problema dell'accettazione è decidibile, ma per le MdT non lo è. Tuttavia, alcuni problemi che coinvolgono gli LBA restano indecidibili. Uno di questi è il problema del vuoto $E_{LBA} = \{ \langle M \rangle / M \text{ è un LBA e } L(M) = \emptyset \}$. Per dimostrare che E_{LBA} è indecidibile, diamo una riduzione che utilizza il metodo delle storie di computazione.

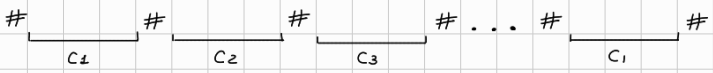
Teorema 5.10

E_{LBA} è indecidibile

IDEA: Questa dimostrazione utilizza una riduzione da A_{TM}. Si dimostra che se E_{LBA} è decidibile, allora lo è anche A_{TM} (contraddizione). Supponiamo che E_{LBA} è decidibile. Come possiamo usare questa assunzione per decidere A_{TM}?

Per una MdT M e un input w, possiamo determinare se M accetta w costruendo un LBA B e poi verificando che $L(B)$ sia vuoto. Il linguaggio che riconosce B comprende tutte le storie di computazione accettanti di M su w. Se M accetta w, tale linguaggio contiene almeno una stringa per cui è diverso dal vuoto. Se M non accetta w, questo linguaggio è vuoto. Se siamo in grado di determinare se il linguaggio di B è vuoto o meno, chiaramente siamo in grado di determinare se M accetta w o meno.

Adesso vediamo come costruire B da M e w. Dobbiamo mostrare come una MdT può ottenere una descrizione di B a partire dalle descrizioni di M e w. Costruiamo B in modo che accetti il suo input x se x è una storia di computazione accettante di M su w. Ricordate che una storia di computazione accettante è una sequenza di configurazioni, $C_1, C_2, \ldots, C_l$ che M occupa quando accetta una qualche stringa w. Ai fini di questa prova si assume che la storia di computazione accettante viene data come una singola stringa con le configurazioni separate l'una dall'altra dal simbolo #, come qui illustrato:



LBA B funziona come segue: quando riceve un input x, B deve accettare se x è una storia di computazione accettante per M su w. In primo luogo, B si ferma su x secondo i delimitatori contenuti nelle stringhe $C_1, C_2 \ldots C_l$. Poi B determina se i vari C_i soddisfano le tre condizioni di una storia di computazione accettante:

1. C_1 è la configurazione iniziale per M su w.
2. Ogni C_{i+1} segue legalmente C_i
3. C_l è una configurazione di accettazione per M.

DIM: Ora siamo pronti a formalizzare la riduzione da A_{TM} a E_{LBA}. Supponiamo che la MdT R decide E_{LBA}. Costruiamo una MdT S che decide A_{TM} come segue:

S = "su input $\langle M, w \rangle$, dove M è una MdT e w è una stringa:

1. Costruisce un LBA B da M e w come descritto nell'idea di dimostrazione;
2. Esegue R su input $\langle B \rangle$;
3. Se R accetta, allora S rifiuta ($L(B)$ è vuoto e quindi M non accetta w)
   Se R rifiuta, allora S accetta ($L(B)$ è non vuoto e quindi M ha una storia di computazione accettante per w)

Quindi, l'esistenza di un decisore per E_{LBA} implica un decisore per A_{TM}. Sappiamo però che A_{TM} è indecidibile, quindi non esiste un decisore per E_{LBA}.

5.2 UN SEMPLICE PROBLEMA INDECIDIBILI

Il fenomeno dell'indecidibilità non è un problema che riguarda solo gli automi. Per mostrare un esempio, consideriamo un problema indecidibile riguardante una "semplice" manipolazione di stringhe: il Problema della Corrispondenza di Post, o PCP.

Possiamo descrivere questo problema paragonandolo ad un tipo di puzzle. Iniziamo con un insieme di tessere del domino, ognuna contenente due stringhe, una su ogni lato. Un domino è del tipo $\left[\frac{a}{ab}\right]$ ed un insieme di domini è del tipo $\left\{\left[\frac{b}{ca}\right],\left[\frac{a}{ab}\right],\left[\frac{ca}{a}\right],\left[\frac{abc}{c}\right]\right\}$. L'obiettivo è quello di costruire una lista di questi domini (le ripetizioni sono ammesse) in modo che la stringa dei simboli scritti nella parte superiore dei domini è uguale a quella dei simboli inferiori. Questa lista è chiamata match. Ad esempio la seguente lista è un match per questo puzzle: $\left\{\left[\frac{a}{ab}\right],\left[\frac{b}{ca}\right],\left[\frac{ca}{a}\right],\left[\frac{a}{ab}\right],\left[\frac{abc}{c}\right]\right\}$

Leggendo l'inizio della stringa otteniamo "abcaaabc", che è uguale alla stringa sotto. Possiamo anche rappresentare questo match in questo modo:

a	b	c	a	a	a	b	c
a	b	c	a	a	a	b	c

Per alcuni insiemi di domini non è semplice trovare un match. Per esempio l'insieme: $\left\{\left[\frac{abc}{ab}\right],\left[\frac{ca}{a}\right],\left[\frac{acc}{ba}\right]\right\}$ non può contenere un match perché ogni stringa che si trova sopra è più lunga rispetto alla stringa che si trova sotto.

Il PCP riguarda la possibilità di determinare se una collezione di domini ha un match. Questo problema non è risolvibile con algoritmi. Formuliamo precisamente il problema e in seguito definiamo il suo linguaggio.

Un'istanza del PCP è un insieme di domini $P=\left\{\left[\frac{t_1}{b_1}\right],\left[\frac{t_2}{b_2}\right],\dots,\left[\frac{t_k}{b_k}\right]\right\}$, e un match è una sequenza $i_1,i_2,\dots,i_l$ dove $t_1t_2\dots t_k = b_1b_2\dots b_k$. Il problema è determinare se P ammette un match. Sia:

PCP = $\{\langle P\rangle \mid P$ è un'istanza del Problema della Corrispondenza di Post che ammette un match$\}$

Teorema 5.15

PCP è indecidibile.

DIM: Per la prova riduciamo A_{TM} al PCP attraverso le storie di computazione accettanti.

Dati $\langle M,w\rangle$ costruiamo un insieme P di dominio nel modo seguente: iniziamo con un domino che ha nella parte di sotto la configurazione iniziale di M e sopra una stringa vuota. Aggiungiamo a P dei domino la cui parte superiore corrisponde alla configurazione corrente e la parte inferiore alla configurazione successiva. Questi domino devono anche tenere conto di:

1. Il movimento della testina;
2. Aumentare lo spazio del nastro con degli spazi vuoti (quando richiesto)
3. Forzare ad usare come primo domino della soluzione del match quello corrispondente alla prima configurazione iniziale di M.

Per semplicità assumiamo che ogni soluzione possibile deve sempre iniziare con il primo domino. Chiameremo il PCP con questa regola aggiuntiva "PCP modificato" o semplicemente MPCP.

Parte 1: Dati $\langle M,w\rangle$, costruiamo il primo domino dell'insieme P' come corrispondente alla configurazione iniziale nel modo seguente:

$\left[\frac{\#}{\#q_0w_1w_2\dots w_n\#}\right]$ =>

#	
#	q_0 w_1 w_2 ... w_n #

Il simbolo # è un nuovo simbolo non appartenente all'alfabeto del nastro di M e serve come marcatore

Step 2 (gestire il movimento della testina a destra): Per ogni $a,b \in \Gamma$ e ogni $q,r \in Q$ dove $q \neq q_{reject}$,

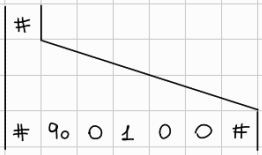
Se $\delta(q,a) = (r,b,R)$, inserisci $\left[\frac{qa}{br}\right]$ in P'

Step 3 (gestire il movimento della testina a sinistra): Per ogni $a,b,c \in \Gamma$ ed ogni $q,r \in Q$ dove $q \neq q_{reject}$,

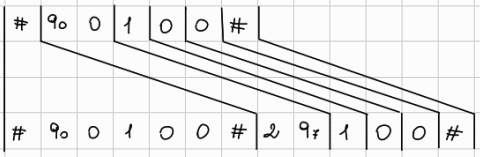
Se $\delta(q,a) = (r,b,L)$, inserisci $\left[\frac{cqa}{rcb}\right]$ in P'

Step 4: Per ogni $a \in \Gamma$, inserisci $\left[\frac{a}{a}\right]$ in P'.

Ora creiamo un esempio ipotetico per illustrare quello che abbiamo costruito finora. Sia $\Gamma = \{0,1,2,\sqcup\}$. Diciamo che w è la stringa 0100 e che lo stato iniziale di M è q_0. Nello stato q_0, leggendo 0, supponiamo che la funzione di transizione dice che M va nello stato q_7, scrive due e muove la testina a destra. In altre parole $\delta(q_0,0) = (q_7,2,R)$. La parte 1 inserisce il dominio $\left[\frac{\#}{\#q_0 0 1 0 0 \#}\right]$ in P' e il match inizia.



Poi la parte due inserisce il dominio $\left[\frac{q_0\,0}{2\,q_7}\right]$ poiché $\delta(q_0,0) = (q_7,2,R)$ e la parte 4 inserisce i domino $\left[\frac{0}{0}\right], \left[\frac{1}{1}\right], \left[\frac{2}{2}\right]$, e $\left[\frac{\sqcup}{\sqcup}\right]$ in P, poiché 0, 1, 2 e $\sqcup$ sono elementi di Γ. Insieme alla parte 5, ciò ci permette di estendere il match a

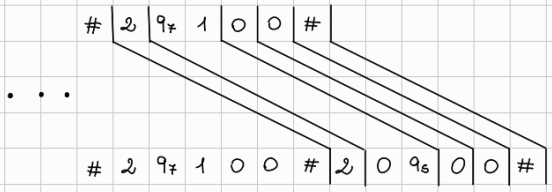


Quindi le tessere delle parti 2, 3 e 4 ci permettono di estendere il match aggiungendo la seconda configurazione dopo la prima. Vogliamo che questo processo continui, con l'aggiunta della terza configurazione, poi della quarta e così via. Affinché ciò avvenga, abbiamo bisogno di aggiungere una o più domini per copiare il simbolo #.

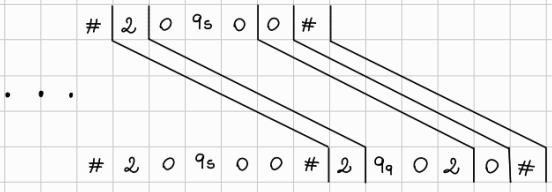
Step 5: Inserisci $\left[\frac{\#}{\#}\right]$ e $\left[\frac{\#}{\sqcup\#}\right]$ in P'.

Il primo domino ci permette di copiare il simbolo # che separa le configurazioni. Inoltre, il secondo domino ci permette di aggiungere il simbolo $\sqcup$ alla fine della configurazione così da simulare gli infiniti blank sulla parte destra del nastro che sono soppressi quando scriviamo la configurazione.

Continuiamo con l'esempio, diciamo che nello stato q_7, leggendo 1, M va nello stato q_5, scrive 0 e muove la testina a destra. Cioè $\delta(q_7,1) = (q_5,0,R)$. Quindi abbiamo il dominio $\left[\frac{q_7\,1}{0\,q_5}\right]$ in P'. Di conseguenza il match parziale viene esteso a:



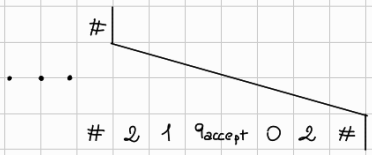
Ora supponiamo che nello stato q_5, leggendo 0 M si sposta nello stato q_9, scrive 2, e muove la testina a sinistra. Cioè $\delta(q_5,0) = (q_9,2,L)$. Quindi abbiamo i domini $\left[\frac{0\,q_5\,0}{q_9\,0\,2}\right], \left[\frac{1\,q_5\,0}{q_9\,1\,2}\right], \left[\frac{2\,q_5\,0}{q_9\,2\,2}\right]$, e $\left[\frac{\sqcup\,q_5\,0}{q_9\,\sqcup\,2}\right]$. Il match viene esteso a:



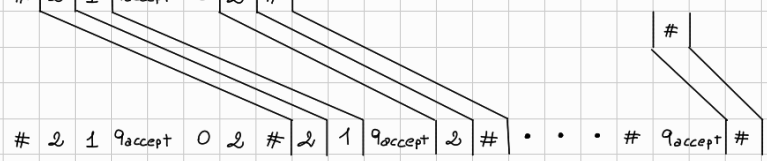
Questo processo continua fino a quando M raggiunge uno stato di arresto. Se tale stato è di accettazione, vogliamo che la stringa superiore venga a coincidere con quella inferiore così da completare il match. Possiamo far sì che ciò accada aggiungendo alcune tessere.

Step 6: Per ogni $a \in \Gamma$, inserisci $\left[\frac{a\, q_{accept}}{q_{accept}}\right]$ e $\left[\frac{q_{accept}\, a}{q_{accept}}\right]$ in P'.

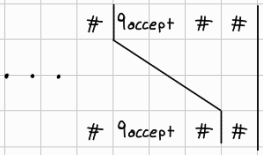
Continuando con l'esempio, se il match parziale quando la macchina si ferma nello stato di accettazione è



I domini appena aggiunti permettono di continuare il match:



Step 7: Infine aggiungiamo il dominio $\left[\frac{q_{accept}\,\#\,\#}{\#}\right]$ e completiamo il match:



Questo conclude la costruzione di P'. Ricordate che P' è un'istanza di MPCP dove il match simula la computazione di M su w. Ora mostriamo come trasformare P' in P, un'istanza di PCP che ancora simula M su w. L'idea è di utilizzare la richiesta inizi con il primo dominio ed inglobarla direttamente nell'istanza del problema così che sia automaticamente soddisfatta.

Introduciamo la notazione necessaria all'implementazione di tale idea.
Sia $u = u_1 u_2 \dots u_n$ una stringa di lunghezza n. Definiamo $\star u$, $u\star$ e $\star u \star$ come le tre stringhe:

$$\star u = *u_1 * u_2 * u_3 \dots * u_n$$
$$u\star = u_1 * u_2 * u_3 * \dots * u_n *$$
$$\star u \star = *u_1 * u_2 * u_3 * \dots * u_n *$$

Per trasformare P' in P procediamo come segue.
Se P' è l'insieme $\left\{\left[\frac{t_1}{b_1}\right], \left[\frac{t_2}{b_2}\right], \left[\frac{t_3}{b_3}\right], \dots, \left[\frac{t_k}{b_k}\right]\right\}$, prendiamo P come segue:

$$\left\{\left[\frac{\star t_1}{\star b_1 \star}\right], \left[\frac{\star t_1}{b_1 \star}\right], \left[\frac{\star t_2}{b_2 \star}\right], \left[\frac{\star t_3}{b_3 \star}\right], \dots, \left[\frac{\star t_k}{b_k \star}\right], \left[\frac{*\diamond}{\diamond}\right]\right\}$$

Vediamo che l'unico dominio che potrebbe essere quello iniziale del match è proprio il primo poiché è il solo in cui sia la parte superiore sia inferiore iniziano con lo stesso simbolo - precisamente $*$. Il dominio $\left[\frac{*\diamond}{\diamond}\right]$ è presente per permettere alla parte superiore di aggiungere l'ulteriore $*$ alla fine del match.

5.3 RIDUCIBILITÀ MEDIANTE FUNZIONI

In questa sezione formalizziamo il concetto di riducibilità. La nozione di ridurre un problema a un altro può essere definita formalmente in vari modi. La scelta di quale usare dipende dall'applicazione. La nostra scelta ricade su un tipo semplice di riducibilità chiamato *riducibilità mediante funzioni*.

FUNZIONI CALCOLABILI

Una MdT calcola una funzione iniziando con l'input della funzione sul nastro e terminando con l'output della funzione sul nastro.

Definizione 5.17
Una funzione $f: \Sigma^* \longrightarrow \Sigma^*$ è una *funzione calcolabile* se esiste una macchina di Turing M che, su qualsiasi input w, si ferma avendo solo $f(w)$ sul nastro.

DEFINIZIONE FORMALE DI RIDUCIBILITA' MEDIANTE FUNZIONI

Definizione 5.20
Un linguaggio A si dice *riducibile mediante funzione* al linguaggio B, e si denota $A \leq_m B$, se esiste una funzione calcolabile $f: \Sigma^* \longrightarrow \Sigma^*$, dove per ogni w,

$$w \in A \iff f(w) \in B$$

la funzione f è chiamata *riduzione* da A a B.

Una riduzione mediante funzione da A a B fornisce un modo per convertire problemi di appartenenza in A in problemi di appartenenza in B. Per verificare se $w \in A$ usiamo la riduzione f per mappare w in $f(w)$ e verifichiamo se $f(w) \in B$ o meno.

Teorema 5.22
Se $A \leq_m B$ è decidibile, allora A è decidibile.

DIM: Sia M il decisore per B ed f la riduzione da A a B. Descriviamo un decisore per A come segue:

N = "su input w:
1. Computa $f(w)$.
2. Esegue M su input $f(w)$ e restituisce lo stesso output di M."

Ovviamente se $w \in A$, allora $f(w) \in B$ perché f è una riduzione da A a B. Quindi, M accetta $f(w)$ ogni volta che $w \in A$. Quindi N svolge il compito desiderato.

Corollario 5.23
Se $A \leq_m B$ è indecidibile, allora B è indecidibile.

Teorema 5.28
Se $A \leq_m B$ e B è Turing-riconoscibile, allora A è Turing-riconoscibile.

La dimostrazione è uguale a quella che abbiamo fatto per il teorema 5.22, tranne per il fatto che M e N sono riconoscitori invece di decisori.

Corollario 5.29
Se $A \leq_m B$ e A non è Turing-riconoscibile, allora B non è Turing-riconoscibile.

Un tipica applicazione di questo corollario, supponiamo che A sia $\overline{A_{TM}}$, il complemento di A_{TM}. Dal Corollario 4.23, sappiamo che non è Turing-riconoscibile. La definizione di riduzione mediante funzione implica che $A \leq_m B$ ha lo stesso significato di $\overline{A} \leq_m \overline{B}$. Per dimostrare che B non è riconoscibile possiamo mostrare che $\overline{A_{TM}} \leq_m \overline{B}$.

Teorema 5.30

EQ_{TM} non è né Turing-riconoscibile né coTuring-riconoscibile

DIM: Come primo passo dimostriamo che EQ_{TM} non è Turing-riconoscibile. Lo facciamo dimostrando che A_{TM} è riducibile a $\overline{EQ_{TM}}$. La funzione di riduzione f opera come segue:

F = " su input $\langle M, w \rangle$, dove M è una MdT e w è una stringa:

1. Costruisce le seguenti due macchine, M_1 e M_2.
   M_1 = "su ogni input:
   1. Rifiuta"
   M_2 = "su ogni input:
   1. Esegue M su w. Se accetta, accetta".
2. Restituisce $\langle M_1, M_2 \rangle$ "

Qui M_1 non accetta alcuna stringa. Se M accetta w, M_2 accetta qualsiasi stringa, quindi le due macchine non sono equivalenti. Viceversa, se M non accetta w, M_2 non accetta alcuna stringa, ed esse sono equivalenti. Perciò f riduce A_{TM} a $\overline{EQ_{TM}}$, come desiderato.

Per dimostrare che $\overline{EQ_{TM}}$ non è coTuring-riconoscibile diamo una riduzione da A_{TM} al complemento di $\overline{EQ_{TM}}$, cioè EQ_{TM}. In questo modo dimostriamo che $A_{TM} \leq_m EQ_{TM}$. La seguente MdT G calcola la funzione di riduzione g.

G = " su input $\langle M, w \rangle$, dove M è una MdT e w è una stringa:

1. Costruisce le seguenti due macchine M_1 e M_2:
   M_2 = "su ogni input:
   1. Accetta "
   M_1 = " Su ogni input:
   1. Esegue M su w;
   2. Se accetta, accetta".
2. Restituisce $\langle M_1, M_2 \rangle$ "

La sola differenza tra f e g si trova nella macchina M_1. In f, M_1 rifiuta sempre, mentre in g accetta. Sia in f che in g, M accetta w se M_2 accetta ogni input. In g M accetta w se M_1 ed M_2 sono equivalenti. Questo è il motivo per cui g è una riduzione da A_{TM} a EQ_{TM}.

Teorema 5.13

ALL_{CFG} è indecidibile

DIM: La dimostrazione procede per assurdo. E' una riduzione da A_{TM} attraverso storie di computazione, ma modifichiamo leggermente la rappresentazione della storia di computazione. Ora descriviamo una procedura di decisione per ALL_{CFG} per decidere A_{TM}.

Per una MdT M ed un input w costruiamo una CFG G che genera tutte le stringhe se e solo se M non accetta w. Così, se M non accetta w, G non genera una particolare stringa. Questa stringa è la storia di computazione accettante per M su w. Cioè G è progettata in modo da generare tutte le stringhe che non sono una storia di computazione accettante per M su w. Per far sì che G generi tutte le stringhe che non costituiscono una storia di computazione accettante per M su w, utilizziamo la seguente strategia.

Una stringa può non essere una storia di computazione accettante per vari motivi. Una storia di computazione accettante per M su w è del tipo $\#C_1\#C_2\#\ldots\#C_i\#$, dove C_i è la configurazione di M al passo i-mo di computazione su w. Quindi G genera tutte le stringhe

1) Che non iniziano con C_1,
2) Che non terminano con una configurazione di accettazione, o
3) in cui qualche C_i non produce C_{i+1} in accordo alle regole di M.

Ora arriviamo alla effettiva costruzione di G. Invece di costruire G, costruiamo un PDA D. Sappiamo di poter convertire D in una CFG. In questa istanza, D partirà in maniera non deterministica per indovinare quale delle tre precedenti condizioni verificare. Una ramificazione controlla se l'inizio della stringa di input è C_1 e accetta se non lo è. Un'altra

ramificazione controlla se la stringa di input termina con una configurazione che contiene lo stato di accettazione, q_{accept}, e accetta se non lo è. La terza ramificazione dovrebbe accettare se qualche C_i non produce correttamente C_{i+1}. Funziona scandendo l'input fino a quando non decide in maniera non deterministica che è arrivata a C_i. Poi inserisce C_i nella pila fino a quando non arriva alla fine come contrassegnato dal simbolo #. Quindi D estrae dalla pila per fare un controllo con C_{i+1}. D accetta se individua una mancata corrispondenza o un aggiornamento improprio.

Il problema con questa idea è che, quando D estrae C_i dalla pila, è in ordine inverso e non adatto al confronto con C_{i+1}. Scriviamo dunque le storie di computazioni accettanti in modo diverso. Ogni seconda configurazione viene scritta al contrario. Le posizioni dispari rimangono scritte nell'ordine originale.